□ PyTorch & Derin Öğrenme Master Mülakat Rehberi

Teorik Temeller, PyTorch Kod Mimarisi ve Sektörel Teknik Mülakat Soru-Cevap Bankası

1. Tenzörler, Boyutlar ve Donanım Mimarisi

Tenzör (Tensor): PyTorch'un temel veri yapısıdır. Çok boyutlu bir sayı dizisidir; GPU (MPS/CUDA) üzerinde paralel matematik çalıştırabilir ve Autograd mekanizmasıyla otomatik türev alır.

Kavram / Operasyon	Açıklama & Mülakat Püf Noktası	Örnek PyTorch Kodu
Shape & ndim	Tenzörün eksen sayısı ve boyutlarıdır. Katman uyumu için shape takibi şarttır.	tensor.shape, tensor.ndim
unsqueeze(1)	1. indekse yeni bir boyut ekler. (N,) şeklindeki 1D vektörü (N, 1) şeklindeki 2D sütun matrisine çevirir.	y.unsqueeze(1)
squeeze()	Boyutu 1 olan gereksiz eksenleri siler. (1, 100, 1) -> (100,).	tensor.squeeze()
view() vs reshape()	view() bellekte yeni kopya açmaz, aynı veriyi paylaşır (hızlıdır). reshape() gerekirse kopyalar.	t.view(2, 5), t.reshape(2, 5)
GPU -> CPU Kuralı	GPU/MPS belleğindeki tenzör doğrudan NumPy'a çevrilemez. Önce CPU RAM'ine çekilmelidir.	tensor.cpu().numpy()

2. Model Mimarisi (`nn.Module` ve `nn.Linear`)

PyTorch'ta tüm sinir ağları nn.Module sınıfından türer. super().__init__() yazılarak ana sınıfın parametre takip yetenekleri devralınır.

Doğrusal Katman Formülü: $Y = X \cdot W^T + b$
• in_features: Katmana giren özellik (feature) sayısı.
• out_features: Katmandan çıkan nöron sayısı.
• Ağırlık (Weight) Boyutu: (out_features, in_features)
• Bias Boyutu: (out_features,)

```
class ClassificationModel(nn.Module): def __init__(self): super().__init__() self.layer_1 = nn.Linear(in_features=2, out_features=10) # 2 girdi -> 10 nöron self.relu = nn.ReLU() # Doğrusalsızlık self.layer_2 = nn.Linear(in_features=10, out_features=1)# 10 nöron -> 1 lojit def forward(self, x): return self.layer_2(self.relu(self.layer_1(x)))
```

3. Aktivasyon Fonksiyonları & Vanishing Gradient

Neden Aktivasyon Şarttır? İki doğrusal katmanın arka arkaya çarpımı (W2 · (W1 · X + b1) + b2) matematiksel olarak yine tek bir doğrusal katmandır (W_eff · X + b_eff). Aktivasyon fonksiyonu olmadan 100 katman da eklense model sadece düz bir çizgi çizebilir.

Fonksiyon	Aralık / Formül	Kullanım Yeri & Püf Noktası
ReLU	max(0, x) Türev: Pozitifte 1, Negatifte 0	Gizli katmanların kralıdır. İşlem yükü sıfırdır, gradyan kaybolmasını önler.
Sigmoid	$1 / (1 + e^{-x})$ Aralık: (0, 1), Türev maks: 0.25	İkili sınıflandırma çıkışında kullanılır. Gizli katmanlarda Vanishing Gradient'e yol açtığı için KULLANILMAZ.
Softmax	$e^{zi} / \Sigma e^{zj}$ Toplamı 1 olan dağılım	Çoklu sınıflandırma çıkışında kullanılır. Tüm sınıfların olasılık toplamını %100 yapar.

4. Tahmin Zinciri: Logits -> Sigmoid -> Round

1. Raw Logits (-∞, +∞): Modelin son katmanından çıkan ham skorlardır.
2. Prediction Probabilities [0.0, 1.0]: torch.sigmoid(logits) ile olasılığa çevrilir.
3. Prediction Labels {0, 1}: torch.round(probs) ile 0.5 eşiğine göre yuvarlanır (≥0.5 -> 1, <0.5 -> 0).

5. Kayıp Fonksiyonları (Loss Functions)

• `nn.MSELoss()`: Regresyon (ev fiyatı, sıcaklık tahmini). Hataların karesini cezalandırır.
• `nn.BCELoss()`: İkili sınıflandırma. Girdi olarak önceden Sigmoid uygulanmış olasılıkları ister.
• `nn.BCEWithLogitsLoss()`: İkili sınıflandırmada altın standarttır. Ham lojitleri alır; Sigmoid ve Cross Entropy'yi Log-Sum-Exp tekniğiyle tek adımda çözerek sayısal taşmayı (underflow/overflow) önler (Sayısal Kararlılık).

6. PyTorch 5-Adımlı Standart Eğitim Döngüsü (Training Loop)

Adım	PyTorch Kodu	Görevi ve Arka Plan Mantığı
0. Mod	model.train()	Modeli eğitim moduna alır. Dropout ve BatchNorm'u etkinleştirir.
1. Forward	y_pred = model(X)	İleri yayılım: Girdi katmanlardan geçer ve tahmin/lojiti üretir.
2. Loss	loss = loss_fn(y_pred, y)	Tahmin ile gerçek etiket arasındaki hatayı skaler olarak hesaplar.
3. Zero Grad	optimizer.zero_grad()	Zorunlu Adım! Autograd gradyanları üst üste topladığı için eski türevleri sıfırlar.
4. Backward	loss.backward()	Geri yayılım: Zincir Kuralı (Chain Rule) ile her ağırlığın gradyanını (∂L/∂W) hesaplar.
5. Step	optimizer.step()	Ağırlıkları günceller: W_yeni = W_eski - (öğrenme_oranı * gradyan).

7. Test / Değerlendirme Modu (Inference Mode)

model.eval() -> Dropout'u kapatır, BatchNorm'u dondurur.
with torch.inference_mode(): -> Autograd türev takip ağacını devre dışı bırakır. Bellek (VRAM/RAM) tüketimini minimize eder ve testi hızlandırır.

8. Overfitting (Aşırı Öğrenme) ve Çözümleri

Overfitting Belirtisi: Train Loss sürekli düşerken (örn: 0.05), Test Loss yükselir veya yüksek kalır (örn: 0.85). Model veriyi ezberlemiştir, genelleme yapamaz.

• Dropout (`nn.Dropout(p=0.2)`): Eğitim sırasında nöronların %20'sini rastgele geçici kapatır. Ağın tek bir nörona bağımlı kalmasını önler.
• L2 Düzenlileştirme (Weight Decay): Ağırlıkların aşırı büyümesini cezalandırır (Adam(..., weight_decay=0.01)).
• Adam Optimizer: Momentum (ivme) ve RMSprop (adapte hız) birleşimidir. SGD'ye kıyasla yerel minimum çukurlarından kolayca çıkar.

□ TEKNİK MÜLAKAT SORU & CEVAP BANKASI (TOP 10)

Sektörel yapay zeka mühendisliği mülakatlarında karşınıza çıkacak en kritik 10 soru ve kıdemli mühendis standartlarındaki ideal cevapları:

Soru 1: Çok katmanlı bir ağda aktivasyon fonksiyonu kullanmazsak ne olur?

İki veya daha fazla doğrusal dönüşümün bileşkesi yine tek bir doğrusal dönüşümdür (W2·W1 = W_eff). Dolayısıyla aktivasyon fonksiyonu olmayan derin bir ağ, matematiksel olarak tek bir doğrusal katmandan farksızdır ve sadece doğrusal karar sınırları (düz çizgi) çizebilir.

Soru 2: Gizli katmanlarda Sigmoid yerine neden ReLU tercih edilir? Vanishing Gradient nedir?

Sigmoid fonksiyonunun türevi maksimum 0.25'tir. Derin ağlarda geriye doğru zincir kuralı uygulandıkça bu türevler birbiriyle çarpılır ve hızla sıfıra yaklaşır. Buna Vanishing Gradient denir; ilk katmanların ağırlıkları güncellenemez. ReLU ise pozitif bölgede 1 türeve sahip olduğu için gradyan kaybolmasını çözer ve işlem yükü sıfırdır.

Soru 3: PyTorch eğitim döngüsünde `optimizer.zero_grad()` neden çağrılmak zorundadır?

PyTorch Autograd motoru `loss.backward()` çağrıldığında gradyanları değişkenin `.grad` alanında otomatik olarak biriktirir (accumulate eder). Eğer `zero_grad()` ile sıfırlanmazsa, önceki epoch'ların türevleri yeni türevlerin üzerine eklenir; bu da gradyan patlamasına ve optimizasyon yönünün sapmasına yol açar.

Soru 4: `nn.BCELoss` ile `nn.BCEWithLogitsLoss` arasındaki teknik fark nedir?

`nn.BCELoss` girdi olarak önceden Sigmoid fonksiyonundan geçirilmiş olasılıkları [0, 1] bekler. `nn.BCEWithLogitsLoss` ise ham lojitleri bekler. İçerisinde Sigmoid ve Binary Cross Entropy işlemlerini Log-Sum-Exp tekniğiyle tek matematiksel adımda birleştirdiği için sayısal taşmayı (underflow/overflow) önler ve sayısal olarak kararlıdır.

Soru 5: `torch.inference_mode()` ile `torch.no_grad()` arasındaki fark nedir?

`inference_mode()`, PyTorch 1.9 ile gelen ve `no_grad()`ın yerini alan daha modern bir mekanizmadır. Yalnızca gradyan takibini kapatmakla kalmaz, tensör versiyon takibini de devre dışı bırakarak daha yüksek hız ve daha düşük bellek tüketimi sağlar.

Soru 6: Sınıflandırmada 'Raw Logit' ne demektir ve sınıf etiketine nasıl dönüştürülür?

Raw Logit, modelin en son doğrusal katmanından çıkan, henüz herhangi bir olasılık fonksiyonundan (Sigmoid/Softmax) geçirilmemiş, (-∞, +∞) aralığındaki kısıtlanmamış ham skordur. Önce `torch.sigmoid(logits)` ile [0, 1] olasılığa, ardından `torch.round(probs)` ile 0.5 eşiğine göre {0, 1} sınıf etiketine dönüştürülür.

Soru 7: Bir tenzörün boyutunu `(800,)` den `(800, 1)` e neden çevririz (`unsqueeze(1)`)?

`(800,)` 1 boyutlu bir vektördür. PyTorch katmanları (`nn.Linear`) ve kayıp fonksiyonları matris işlemleri yaptığı için verinin 2 boyutlu `(batch_size, num_features)` formatında olmasını zorunlu kılar. `unsqueeze(1)` 1. eksene boyut ekleyerek shape'i (800, 1) yapar.

Soru 8: Dropout katmanı test modunda (`model.eval()`) nasıl davranır?

Dropout eğitim esnasında nöronların belirtilen oranını (örn: %20) rastgele sıfırlayarak ağın aşırı ezberlemesini önler. Ancak test/tahmin esnasında (`model.eval()`) tüm nöronlar tam kapasiteyle aktiftir; nöron kapatma işlemi otomatik olarak devre dışı bırakılır.

Soru 9: Apple Silicon (MPS) veya CUDA üzerindeki tensör neden doğrudan `.numpy()` yapılamaz?

GPU/MPS belleğindeki tensörler PyTorch Autograd grafiğine bağlıdır ve ekran kartı VRAM'inde tutulur. NumPy ise saf C tabanlı bir kütüphanedir ve yalnızca CPU ana belleğindeki (RAM) verilerle çalışabilir. Bu nedenle önce `.cpu()` ile ana belleğe çekilmeli, ardından `.numpy()` çağrılmalıdır.

Soru 10: Adam optimizer'ı SGD'den üstün kılan iki temel mekanizma nedir?

Adam, Momentum ve RMSprop algoritmalarının birleşimidir. Momentum sayesinde önceki gradyanların hareketli ortalamasını tutarak fiziksel bir ivme kazanır ve yerel minimum çukurlarından çıkar. RMSprop sayesinde ise her bir parametrenin güncelleme hızını kendi gradyan büyüklüğüne göre otomatik olarak ayarlar.